



REMORA: Real-time Expressive Motion to Output Routing Audio

Timothée Dravet

Polytech Marseille, Aix-Marseille Université
France
timothee.dravet@etu.univ-amu.fr

Daniel Overholt

CREATE, Aalborg University
Denmark
dano@create.aau.dk

Abstract

REMORA turns a martial arts staff (bō) into a musical instrument: spins, tilts, and strikes become sound in real time. A wireless sensor unit on the staff tracks its motion and sends it to a companion audio plugin, which we built around a simple idea - instead of hard-wiring one fixed gesture-to-sound behaviour, the mapping itself is a small patch of connected building blocks that the performer can see and rewire live, on a visual canvas inside the plugin, much like a modular synthesiser. We show that this same small set of building blocks is enough to reproduce, as editable patches, everything from simple tilt-controlled pitch to compound movement-quality mappings inspired by Laban Movement Analysis. This paper describes the instrument’s hardware, its self-calibration method, the mathematics behind turning raw orientation into musical control, and the live patching interface itself. We situate it against prior staff-shaped, baton, and visual-patching instruments. Grounded in implementation-level evidence, together with a protocol for the performer evaluation still to be run, we reflect on what it means to make an instrument’s gestural vocabulary an explicit, editable part of the instrument.

CCS Concepts

- **Applied computing** → **Sound and music computing**;
- **Software and its engineering** → *Visual languages*.

Keywords

digital musical instrument, gestural control, inertial sensing, Laban Movement Analysis, OSC, staff instrument

1 Introduction

1.1 Motivation and Background

REMORA grew out of the first author’s parallel practice in martial arts and music, and a frustration shared by many hand-built digital musical instruments: once a gesture vocabulary is designed and coded, changing it means rebuilding the instrument rather than performing with it. That friction is the paper’s real starting point - not which single mapping best suits a staff, but whether a staff instrument’s gestural vocabulary could instead be something the performer authors and revises live, the way a modular-synthesis patch is built and rebuilt on a rack.

The bō staff itself is a form factor that New Interfaces for Musical Expression (NIME) instruments have rarely explored: a full-body, swung, two-handed controller with its own rotational inertia and reach, distinct from the small handheld or body-mounted objects most inertial-sensor Digital Musical Instruments (DMIs) use. Pairing that distinctive controller form with a mapping layer that can

be reshaped at performance time, rather than fixed at design time, is the combination this paper investigates.

1.2 Objectives of the Study

This article supports four claims, each grounded in the system as it currently operates:

- (1) A small dataflow language - source, math, and sink node primitives - is embedded directly in the audio plugin and evaluated once per audio block as a directed acyclic graph (DAG); the performer rewires it live, without recompiling any code.
- (2) That same handful of primitives is expressive enough to reproduce eleven previously hand-coded mapping strategies as ordinary presets: the mapping design space explored by hand over the course of the project turns out to be spanned by a fairly compact algebra of building blocks.
- (3) A magnetometer-free, world-frame heading estimate (“facing”) is derived purely from the calibrated orientation stream, eluding the magnetic interference a stage rig would otherwise cause - a claim we bound to session lengths on the order of tens of minutes, since uncorrected gyroscopic yaw drift is expected to accumulate over longer, uninterrupted play (§6.1).
- (4) A mount-agnostic calibration transform lets the same instrument logic run correctly regardless of where the sensor is physically mounted on the staff.

1.3 Scope and Applications

The instrument’s intended context follows directly from its origin: a solo performer with some staff or martial-arts background, playing a set built from one or more presets rather than a fixed, single-piece score. The preset catalog (Table 2) is deliberately graded rather than uniform - *Simple* asks only for basic tilt and roll, while the Laban-driven *Martial* and *Azimuth* presets reward staff-handling vocabulary the performer already has from bo practice - so the same instrument can serve a newcomer exploring gesture-to-sound relationships and a martial artist bringing an existing physical vocabulary onto a stage. Because the mapping itself is authored in the same visual language a modular-synthesis performer would recognise, we also see REMORA as being of interest to patch-based electronic musicians looking for a full-body controller to wire into an existing practice, not only to staff practitioners looking for a way into live electronics. We do not, however, claim REMORA is suited to ensemble or accompanied performance in its current form: the single-IMU, single-staff design in §6.1 means the instrument has no notion of a second performer’s motion, and no mapping in the current catalog was designed with an ensemble context in mind.



This work is licensed under a Creative Commons Attribution 4.0 International License.

2 Related Work

2.1 Staff, Rod, and Baton DMIs

Rod-, staff-, and baton-shaped controllers form a recurring family of DMIs, spanning handheld sensor-augmented rods, conducting tools, and swung objects.

T-Stick [6] - a family of rod-shaped controllers sensing finger position and pressure along the shaft via capacitive touch, together with accelerometer-derived shake and tap gestures. The sensing is paired with an external, reconfigurable mapping layer built around the digital-orchestra/libmapper tools, so the gesture vocabulary can be revised without changing the hardware.

Digital Baton [7] - Marrin and Paradiso’s controller, combining accelerometers, infrared position tracking, and a pressure-sensitive tip to capture continuous conducting gesture rather than discrete note events.

Conductor’s Jacket [8] - a wearable, full-body electromyography (EMG) and motion rig extending the same conducting-gesture goal to the whole body, recording muscle tension and movement alongside the baton’s hand-level detail.

Musical Poi (mPoi) [9] - Nam’s sensor-augmented poi, generating sound from swing motion; a compact, single-mapping instrument that established spun/swung objects as a recognised, if uncommon, DMI form.

2.2 Inertial Sensing and Visual Patching in NIME

Visual dataflow patching and Laban-derived movement analysis are two long-running, separate threads in NIME and related work: one concerned with how a mapping is authored and rewired, the other with what expressive qualities a mapping can draw on.

Max [10] - originated as Puckette’s Patcher and has been maintained and extended commercially for over three decades, a history Puckette himself traces in “Max at Seventeen” [12]. It established the boxes-and-patch-cords model of unit generators wired together and rewired live that much of visual audio programming still follows.

Pure Data [11] - Max’s open-source sibling, built around the same patching model and widely used in NIME instrument-building for its extensibility and free licensing.

VCV Rack [1] - a community-maintained open-source descendant of the same patching idea applied to virtual modular synthesis, letting performers wire together emulated Eurorack-style modules on screen.

EyesWeb [3] - Camurri et al.’s system for extracting expressive movement qualities from full-body motion in real time, grounded in Laban’s Effort theory and built for interactive dance and music applications.

DAMPER [2] - Bennett et al.’s force-feedback interface platform, built on Laban’s four Effort factors as the direct design basis for its haptic response.

Ward et al. [14] - use Laban Effort - Flow and the Recuperation phase of Effort phrasing in particular - as an analytic lens on theremin performers’ movement, to inform how future instruments might be designed around it.

Schacher [13] - argues for Laban-derived motion/effort elements as a systematic basis for gesture-to-sound mapping in interactive dance, rather than an ad hoc or purely descriptive analysis tool.

2.3 Mapping Strategies

Beyond specific instruments, a body of work addresses gesture-to-sound mapping as a design problem in its own right, independent of any single controller’s hardware.

Hunt and Wanderley [5] - categorise gesture-to-synthesis mappings by complexity - one-to-one, one-to-many, many-to-one - and argue that richer, many-to-many mappings are what make an interface feel like an instrument rather than a remote control.

Wekinator [4] - Fiebrink and Cook’s system for authoring a gesture-to-sound relationship via interactive machine learning: a performer or designer supplies example gestures and desired outputs, and regression learns the mapping, without either party writing or wiring an explicit mapping function.

3 Design and System Architecture

3.1 Initial Design Requirements

Five requirements shaped the design from the outset, chosen to keep the instrument performable:

- **Full-body, swingable staff form factor.** Not a handheld controller: the instrument must support the reach, weight, and two-handed grip of an actual *bô* - roughly 1.5 m long - not a miniaturised stand-in for one.
- **Wireless sensing during performance.** A cable is acceptable only for charging between sets; nothing should tether the performer to a fixed point on stage.
- **Live-editable mapping.** Switching or rewiring the mapping graph must not interrupt audio output, so a change can be made mid-rehearsal without stopping the sound.
- **Mount-agnostic calibration.** Instrument behaviour must not depend on precisely where the sensor is glued to the staff, since exact placement varies from build to build and cannot be guaranteed reproducible by hand.
- **Low end-to-end latency.** The staff has to feel driven by the performer’s motion, not lagging behind it, which bounds how much time sensing, transport, and synthesis are together allowed to take.

3.2 Architecture Overview

REMORA’s architecture has two parts: firmware handles sensing and transport, and the plugin handles everything downstream of the incoming Open Sound Control (OSC) stream, from calibration through to the audio it produces. Figure 1 lays out that split directly, left to right: IMU sampling and OSC transmission on the staff, feeding the plugin’s own reception-to-synthesis pipeline on the host.

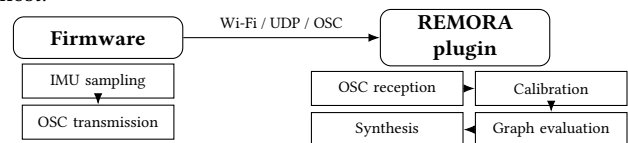


Figure 1: REMORA’s signal flow, from IMU sampling on the staff to synthesis on the host.

4 Implementation

4.1 The Build

The sensor unit sits in a small OpenSCAD-parametrised enclosure, sized to the bâ's diameter. Because calibration is mount-agnostic, it can be placed anywhere along the staff - in practice usually near one extremity, out of the performer's grip - and is secured with zip ties rather than adhesive or drilling, so it can be repositioned or removed freely.

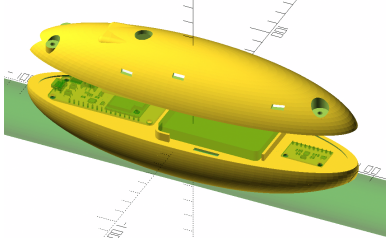


Figure 2: The parametric OpenSCAD enclosure, sized to the staff diameter and secured with zip ties.

4.2 Hardware

Inside the enclosure, a SparkFun ESP32-S2 Thing Plus reads a 9-degrees-of-freedom (9-DOF) MPU-9250 IMU over the Inter-Integrated Circuit (I2C) bus at 100 Hz and packs orientation, acceleration, and gyroscope readings into an OSC packet sent over the UDP on port 8000. The firmware joins the host computer's own Wi-Fi hotspot directly rather than a dedicated access point, which keeps setup to a single pairing step at the cost of the range and interference sensitivity Discussion returns to (§6.1).



Figure 3: Component arrangement inside the enclosure - ESP32-S2, MPU-9250 IMU, and LiPo battery, wired as described in Hardware.

4.3 Software: Calibration and Motion Mathematics

Every equation below comes straight from the running code (optimized in the implementation), each tagged with the function it lives in.

Orientation and Calibration. Rotation. Orientation is tracked as a unit quaternion (no gimbal lock); rotating v by q gives $v' = q v q^{-1}$, with q^{-1} the conjugate. Euler angles are derived only once, at the end, for the handful of mappings that still need them.

[MathHelpers::rotate] [conjugate]
[rotateVectorByQuaternion] [toEuler] [multiply]

Calibration, stage 1 - which axis is "forward". The sensor can be glued onto the staff at any orientation, so each candidate axis $L \in \{\pm X, \pm Y, \pm Z\}$ is scored against three reference poses A, B, C (quaternions q_A, q_B, q_C , chosen so forward varies substantially between them while up/right do not):

$$v_P = \text{rotate}(L, q_P), \quad e(L) = |v_A \cdot v_B| + |v_B \cdot v_C| + |v_C \cdot v_A|$$

A low $e(L)$ (near-orthogonal v_A, v_B, v_C) marks the genuine forward axis; restricted to right-handed candidates,

$$L^* = \arg \min_{L: (v_A \times v_B) \cdot v_C < 0} e(L)$$

gives the alignment quaternion q_{align} .

[computeAlignQuat]

Stage 1 failure modes. computeAlignQuat reports failure when the three poses are too close together or nearly coplanar to fix a physical axis with confidence, and the User Interface (UI) surfaces this as a "Calibration failed, try again" prompt.

Calibration, stage 2 - fixing the twist. The sensor could still be rotated around L^* . The measured forward/up/right triad and the ideal instrument-frame triad are each orthonormalized via Gram-Schmidt into U and W ; the aligning rotation is the orthogonal Procrustes closed form

$$R = W U^T,$$

converted to a quaternion q_{corr} via Shepperd's method.

[computeCorrection] [MathHelpers::fromMatrix]

Runtime. Every live reading becomes instrument space via $q' = q_{\text{corr}} (q_{\text{raw}} q_{\text{align}})$ - independent of where the sensor sits on the staff.

[REMORAProcessor::getCalibratedQuat]

Motion Feature Extraction. Everything below runs once per received packet, inside the shared analyzer that every mapping graph reads from - so no individual mapping repeats this work. The smoothing rates, thresholds, and envelope times named throughout this subsection were tuned by ear against the physical instrument during iterative testing, not derived analytically or fit to recorded data; see Discussion for the resulting lack of a perceptual evaluation of these choices.

[StaffMotionAnalyzer::computeFrame]

Table 1: Per-packet motion features.

Feature	Purpose
Timestep Δt	reject implausible packet gaps
Tip vector / rotation axis	derive tip velocity & spin axis
Gravity / dynamic accel.	separate gravity from motion
Laban Weight	movement forcefulness
Laban Time	suddenness of rotation-speed change
Laban Space	spin-axis consistency (focus)
Laban Flow	bound vs. free movement
Spin classification	vertical/horizontal spin & direction
Facing	east-west vs. north-south heading
Continuous spin count	count full turns per spin
Axial thrust	detect stab/strike along staff axis

Timestep. Δt is the time between two packets, clamped to a 0.1-200 ms valid range so a dropped or duplicated packet can't send the integrators below flying.

[calculateDeltaTime]

Tip vector and rotation axis. The tip direction $p_n = \text{rotate}((1, 0, 0), q_n)$ is buffered over the last three samples to derive tip velocity and, from that, the axis the staff is currently spinning around.

[updateTipPositionHistory] [calculateRotationAxisAtMidpoint]

Gravity and dynamic acceleration. The accelerometer always mixes gravity in with whatever motion the performer is making. To separate the two, REMORA keeps a running estimate of which way “down” currently is, by smoothing the inverse-rotated world +Z axis:

$$g_n = \text{onePole}(g_{n-1}, \text{rotate}(+Z, q_n^{-1}), \alpha=0.10)$$

Subtracting this gravity estimate from the raw reading leaves just the motion: $a_{\text{dyn}} = a_{\text{raw}} - g$.

[updateGravityVector] [calculateDynamicAcceleration]

Laban features. All four Laban Effort qualities reduce to the same shared one-pole smoother,

$$y_n = y_{n-1} + \alpha (x_n - y_{n-1}), \quad \alpha \in (0, 1],$$

applied to a derivative of either dynamic acceleration or gyroscope magnitude: Weight integrates dynamic acceleration into a leaky velocity estimate (its magnitude, fast-attack/slow-release enveloped) to capture forcefulness; Time is the clamped-positive rate of change of raw gyroscope magnitude, capturing suddenness; Space is the smoothed absolute dot product between consecutive rotation-axis directions, capturing how consistently the staff spins around the same axis versus wandering; Flow is the smoothed, clamped jerk of dynamic-acceleration magnitude, split into flow_bound (controlled) and $1 - \text{flow_bound}$ (free).

[MathHelpers::applyOnePoleFilter]
[integrateVelocityForLabanWeight] [updateLabanWeight]
[updateLabanTime] [updateLabanSpace] [updateLabanFlow]

Spin classification. Is the staff spinning like a baton (vertical) or more like a fan blade (horizontal)? Let $(\text{axis}_x, \text{axis}_y, \text{axis}_z)$ be the $\alpha=0.35$ one-pole-smoothed rotation axis from above. The spin is classified vertical when

$$\sqrt{\text{axis}_x^2 + \text{axis}_y^2} \geq |\text{axis}_z|,$$

i.e. when the horizontal-plane component outweighs the z component. Spin direction (clockwise/counter-clockwise, CW/CCW) then comes either from the sign of the dominant in-plane axis component, or - in the Martial mappings - from the sign of that axis’s dot product against a reference heading latched once, the first time a qualifying vertical spin happens after the mapping loads.

[updateSpinClassificationByAbsoluteComponent]
[updateSpinClassificationByReferenceAzimuth]

Facing. Rather than compute a compass heading - which would need a magnetometer, and stages are full of magnetic interference - REMORA asks a simpler question: is the staff pointing more east-west or more north-south? Let $(\text{dir}_x, \text{dir}_y)$ be the horizontal-plane components of whichever direction is currently relevant - the rotation axis during vertical spins, the tip direction during horizontal ones. Facing is then just a hysteresis comparison,

$$|\text{dir}_x| \geq 1.15 \cdot |\text{dir}_y|,$$

where the 1.15 factor gives asymmetric switch points ($\approx 48^\circ/42^\circ$) so the classification doesn’t flicker right at the boundary.

[updateFacingClassification]

Continuous spin count. Full turns in the current spin, from $\|\omega\| \cdot \Delta t$ accumulated to 360° per lap; resets whenever the spin classification changes.

[accumulateContinuousSpins]

Axial thrust detection. Flags a stab/strike when the jerk of the tip-projected dynamic acceleration crosses zero as its magnitude passes 1.5g, gated against fast spins ($\|\omega\| < 90^\circ/\text{s}$) and cooled down between hits.

[detectAxialThrustPeaks]

Sensitivity of tuned constants. Each constant above biases the instrument in a specific direction. The gravity smoother’s $\alpha=0.10$ trades under-reporting fast motion (too high) against a lagging estimate that produces a spurious motion blip when the staff is set down (too low). The spin-axis smoother’s $\alpha=0.35$ plays the same role for Laban Space: higher reads axis noise as “inconsistent” too readily, lower reads a genuinely wandering spin as falsely focused. The facing hysteresis factor (1.15) sets the $\approx 48^\circ/42^\circ$ dead-band; 1.0 reintroduces boundary flicker, while a much larger factor makes facing changes sluggish to register. The thrust detector’s 1.5g threshold and $90^\circ/\text{s}$ gate trade false triggers during forceful spins (too low) against missed soft stabs (too high).

4.4 Software: Mapping Architecture

NodeGraph/NodeTypeRegistry: four node categories - Source (IMU, Laban features), Math (arithmetic, shaping, dynamics, lookup tables), Sink (synth-engine parameters, one resizable “mega-node” per engine exposing every one of that engine’s parameters as its own typed input port, §4.7), and Display (resizable, performer-facing live monitors, §4.6) - each a small typed function registered once. A graph is nodes plus typed wiring, serialised to/from the Extensible Markup Language (XML) as a preset; GraphMappingStrategy makes the graph the only mapping path the synth talks to, with no separate hand-written path.

Graph Evaluation and Primitive Library. DAG evaluation. A patch is a directed graph of typed nodes; NodeGraph::connect provisionally adds an edge and recomputes a topological order with Kahn’s algorithm via NodeGraph::recomputeTopoOrder, reverting the edge if the graph fails to fully drain (a closed cycle). Each audio block, NodeGraph::evaluate walks the cached order once, and every node computes

$$y_i = f_{\text{type}(n_i)}(x_{i,1}, \dots, x_{i,m}; \theta_i)$$

where n_i is the i -th node in topological order, $\text{type}(n_i)$ its registered node-type function, $x_{i,1}, \dots, x_{i,m}$ its wired inputs, θ_i its parameters, and y_i its output(s) - so the whole patch is one composed function of the sensor frame, evaluated once per block with no re-sorting on the audio thread.

Primitive vocabulary. Every node type is one small pure function (or, for stateful ones, a function plus a scratch struct): arithmetic (add/subtract/multiply/divide/abs/max/min/equals), shaping (mapRange, clamp, threshold, crossfade, quantizeSteps, sine, atan2, semitonesToHz), lookup tables (a general 1-D table and the 3-boolean-selector table used by the *Azimuth* root table below), and

stateful dynamics (one-pole smoother, leaky integrator, retrigger envelope, hysteresis step, derivative, latched smoother).

[Graph::buildAllNodes]

4.5 Software: Named Mappings as Graph Presets

Table 2 shows the generalisation claim directly: each of the eleven mappings - *Simple*, *Lead+Drone*, *Spin Filter*, *Bowed Chord*, the *Azimet* family (facing-based root table, spin-count filter sweep, axial thrust transients), *Martial Effort/Martial Momentum*, *Spin Voices*, and *Speed Gate* - is built once via `GraphBuilder` and is structurally just a particular wiring of the same node catalog. Counts are read directly off the constructed graphs (`NodeGraph::nodes()`).

Four further presets - *Wind Noise*, *Azimet Kinetic*, *Spin Voices (Scale)*, and *Vocal Tract* - sit outside that generalisation claim: unlike the eleven above, none of them was ever a separate hand-coded C++ mapping class first: they were authored directly as graph patches, so they demonstrate the catalog’s expressiveness going forward rather than its ability to reproduce a prior hand-built design. *Vocal Tract* is also the one preset that drives the vocal-tract engine instead of the additive one (§4.7). Counting these, the catalog currently ships fifteen presets in total. Across all eleven presets, only 63

Table 2: Node composition of each named-mapping preset.

Preset	Nodes	Source	Math	Sink
Simple	7	2	3	2
Wind Noise	<i>tbd</i>	<i>tbd</i>	<i>tbd</i>	<i>tbd</i>
Bowed Chord	110	25	54	31
Lead+Drone	96	23	45	28
Spin Filter	105	30	54	21
Martial Effort	207	46	130	31
Martial Momentum	109	32	48	29
Azimet	113	32	52	29
Azimet+	113	32	52	29
Azimet Reverb	123	33	58	32
Azimet Kinetic	<i>tbd</i>	<i>tbd</i>	<i>tbd</i>	<i>tbd</i>
Speed Gate	181	87	95	29
Spin Voices	99	34	36	29
Spin Voices (Scale)	<i>tbd</i>	<i>tbd</i>	<i>tbd</i>	<i>tbd</i>
Vocal Tract	<i>tbd</i>	<i>tbd</i>	<i>tbd</i>	<i>tbd</i>

of the catalog’s 79 registered node types are ever instantiated - 42 of those source/math primitives, the rest sink writers. A full development cycle of hand-built mapping design, spanning a solo tilt-controlled mapping through the fully Laban-driven *Martial* and *Speed Gate* patches, lands inside less than two-thirds of an already small catalog; no preset needed a node type invented specifically for it. That result should be read carefully, though: the catalog itself was not fixed in advance and then tested against eleven independent designs - it grew alongside the eleven mappings, several of which were the reason a given primitive was added in the first place. What Table 2 demonstrates is internal consistency (the resulting catalog is compact enough to re-express every mapping it was abstracted from as a preset, rather than needing a bespoke escape hatch per mapping), not that the catalog would prove sufficient for a gesture vocabulary designed independently of it. Whether a mapping designer outside this project can build a genuinely new

movement-quality mapping from the existing 63 node types without hitting a missing primitive is an open question the current evidence does not settle.

[NodeGraph::nodes] [Graph::Presets::AllPresets]

Azimet-Specific Transformations. Shared by every Azimet-family preset.

Root-note table. The spin-plane/direction/facing root-note table above is not an if/else chain - it is literally an 8-entry lookup table (LUT) indexed by three booleans (`isVertical`, `isCCW`, `isFacingEast`) through the graph’s generic 3-selector LUT node.

Root morph. The target semitone value is chased by a variable-rate one-pole filter whose rate is

$$\text{rate} = \max\left(\text{clamp}\left(\frac{\|\omega\|}{2000}, 0.005, 0.2\right), 0.5 \cdot \text{onsetEnv}\right)$$

where `onsetEnv` is a retrigger envelope (decay 0.90) that fires the instant the staff leaves rest - faster motion, or a fresh onset, morphs the root pitch in quicker.

Filter sweep. The spin count drives a phase modulator whose sine output is remapped onto the filter’s cutoff range and smoothed:

$$c_{\text{raw}} = \text{mapRange}(\sin(1.5 \cdot \text{spinCount}), [-1, 1], [400, 20000]\text{Hz})$$

$$\text{cutoff} = \text{onePole}(c_{\text{raw}}, \text{rate}=0.03)$$

where `spinCount` is the unsigned lap counter from Continuous Spin Count above, `mapRange` linearly rescales its argument from the first interval to the second, `craw` the resulting unsmoothed cutoff target, and `onePole` the smoothing primitive defined earlier.

[Graph::Presets::buildAzimetCore] [buildSpinCountLpfHz]

4.6 Graphical Interface: Live Patching as Performance Practice

The node-graph editor is the instrument’s primary authoring surface, not a debug tool: a pannable, zoomable canvas on which each node renders as a coloured card - green for Source, yellow for Sink, purple for Display, neutral for Math - labelled with its category and inline, live-editable parameter fields; typed input/output pins along each card’s edges accept click-and-drag connections rendered as cubic-Bézier wires, colour-highlighted while dragging and hit-testable afterward for rewiring or deletion via context menu, which for an existing wire also offers jumping the view straight to its source or destination node, briefly highlighted on arrival so the jump is easy to follow in a large patch. New nodes are added from a categorised picker at any canvas position, each entry annotated with a short description of its behaviour; a search bar over the same picker jumps directly to a node type by name. An auto-layout pass ranks every node by longest path from a source, pulls sink-only nodes as late as their wiring allows, threads dummy nodes through any connection that spans more than one rank so a long wire does not cut across intervening columns, and orders nodes within each resulting layer to reduce crossings - a layered graph-drawing pass rather than a fixed column assignment, so a freshly loaded preset still untangles left-to-right from source to sink but copes with the deeper, more irregular graphs later presets produce. Hovering any node or pin surfaces its current, live-updating value as a tooltip, and three dedicated Display node types - a numeric readout, a level meter, and a scrolling scope - can be dropped inline on any wire

and freely resized, giving the performer a live, in-patch view of a signal without altering it (all three are pure passthrough: the graph carries exactly the same value with or without one attached). Sink mega-nodes (§4.7) are themselves width-resizable and label each port with its valid range (e.g. [20, 20000] LPF Cutoff), so a wide parameter surface stays legible without a separate details panel; at the opposite end of the size scale, constant-value nodes render as a compact numeric box with no slider, so the canvas is not dominated by its simplest node type. A dirty-state indicator tracks whether the active patch has diverged from its saved preset, with one-click revert - so a performer can improvise a mapping change mid-rehearsal and either keep or discard it, treating the mapping itself as something authored and iterated on at the same level as the sound it produces. Each preset in the catalog also carries a short description, surfaced in the preset selector, so a performer choosing between mappings mid-rehearsal is not reading bare names.

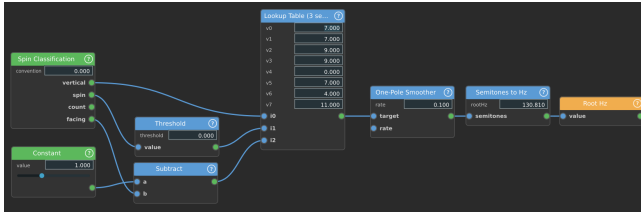


Figure 4: Schematic of the node-graph editor canvas - coloured node cards (green Source, yellow Sink, purple Display, neutral Math) with typed pins, connected by Bézier-rendered wires.

4.7 Software: Synthesis Engine

Every mapping graph writes into a shared `SynthManager`, which owns a small, pluggable set of synthesis engines behind one `ISynthEngine` interface (`prepare/render`) rather than a single hard-coded voice: an additive engine, a granular engine, and a physically modelled vocal-tract engine. Each engine reads its own parameter struct - one field of the shared `MappingOutput` the active graph writes every block, addressed through that engine's sink mega-node (§4.5) - and renders it into its own scratch buffer; `SynthManager::processBlock` sums whichever engines the active preset actually wires in and applies one shared master-gain/mute stage on top, keeping the engines themselves ignorant of which preset or graph is driving them - which is what lets a graph be swapped or rewired live without reinitialising audio. An engine a preset's graph leaves unwired is skipped entirely that block - no parameter update, no render call - rather than rendered silently and discarded, so the additive-only presets that make up most of the catalog never pay the granular or vocal-tract engines' cost.

```
[ISynthEngine] [SynthManager::processBlock]
```

Additive Engine. The original, still default, engine; every preset in Table 2 except *Vocal Tract* drives it.

```
[AdditiveSynthEngine::render] [SynthVoice::tick]
```

Harmonic bank. Each of the 4 voices sums 6 phase-accumulated sine partials at integer multiples of a (vibrato- and chorus-modulated) fundamental, each with its own independently smoothed target amplitude.

Waveshaping. A rational soft-clip saturator, applied per-voice when `driveAmt > 0`:

$$y = \frac{x(1 + \text{driveAmt})}{1 + |x(1 + \text{driveAmt})|}$$

where x is the voice's summed harmonic-bank sample before shaping and `driveAmt` the mapping-supplied drive amount.

Resonator. A short feedback comb (1800-sample delay, 0.35 feedback, mixed 0.7 dry / 0.3 wet) is layered onto every voice for a string-like coloration.

Modulation. Vibrato and tremolo are sinusoidal multipliers on pitch and gain respectively:

$$f' = f(1 + \text{depth} \cdot \sin \phi_v) \quad g' = g(1 - \text{depth} \cdot (0.5 + 0.5 \sin \phi_t))$$

where f, g are the fundamental frequency and master gain before modulation, f', g' the modulated values actually used, and ϕ_v, ϕ_t the vibrato and tremolo Low-Frequency Oscillator (LFO) phase accumulators (radians), each advancing by its own rate every sample. A per-voice phase-offset sine (depth 0.003, rate 0.3 Hz) adds chorus detuning; chord changes cross-fade linearly between old and new semitone offsets over 200 ms rather than jumping.

Noise and filtering. An xorshift32 pseudo-random number generator (PRNG) feeds a one-pole lowpass (coefficient `noiseLpCoef`) before being mixed in at `noiseAmount`; the summed stereo output passes through a JUCE state-variable Topology-Preserving Transform (TPT) lowpass, its cutoff set directly each block from the sink.`additiveSynth` mega-node's *LPF Cutoff* port (only range-clamped, not ramped), and an optional Schroeder-style `juce::dsp::Reverb` send whose wet level is ramped through a per-sample `SmoothedValue` to avoid zippering when the active mapping changes.

Pitch/frequency conversion.

$$f = f_{\text{root}} \cdot 2^{\text{semitones}/12}$$

where f_{root} is the mapping's current root frequency in Hz and `semitones` the signed offset applied to it, giving the equal-tempered result f .

```
[MathHelpers::semitoneRatio] [convertSemitonesToHertz]
```

Granular Engine. Has no audio input and bundles no sample assets: on `prepare` it synthesises its own roughly 2-second internal source buffer (a slowly evolving harmonic bed layered with lightly filtered noise), then continuously spawns and scans up to 64 overlapping grains across that buffer, each with its own position, playback rate (a negative rate plays a grain backward), pan, and amplitude envelope, driven every block by the mapping's granular parameter struct. Silent by default, so a preset that never wires `sink.granularSynth` costs nothing beyond the skipped check above. Currently used by *Wind Noise*.

```
[GranularSynthEngine::buildSourceBuffer]
[GranularSynthEngine::spawnGrain]
```

Vocal-Tract Engine. The one engine not built for this project: an LF-model glottal source excites a digital-waveguide vocal tract with oral and nasal branches, run at 2× oversampling, with two RBJ-cookbook bandpass filters colouring raw white noise into breath and frication noise before it reaches the model. The model itself is a C++ port (via `cutelabnyc/pink-trombone-cpp`, MIT-licensed) of Neil Thapen's *Pink Trombone*; REMORA's contribution at this layer

is the driver around it - control-rate parameter updates from the graph's sink. `pinkTromboneSynth` mega-node each block - not the vocal-tract model itself. It is the one engine so far with a dedicated preset, *Vocal Tract* (§4.5), rather than being an option within an otherwise-additive mapping.

[`PinkTromboneEngine::render`]

4.8 Development Iterations

The node-graph engine above did not arrive fully formed; it is the result of three iterations in which generalisation followed rather than preceded hand-built mapping design. The first was firmware reading the IMU and streaming OSC into a minimal receiver, driving a single hardcoded tilt-to-pitch mapping, purely to prove the sensor-to-sound pipeline end to end. The second built the eleven named mappings one at a time as their own C++ classes, each adding a new gesture vocabulary (chords, Laban Effort, world-frame facing, transient detection) on top of the shared `StaffMotionAnalyzer` - at which point a pattern became visible: every one of them was really just arithmetic, shaping, and a handful of stateful primitives wired together in a fixed pattern. The third iteration rebuilt the mapping layer as the small generic graph engine described above, porting all eleven prior mappings over as presets rather than deleting them.

Since that third iteration, the same graph engine has absorbed two further kinds of growth without a fourth architectural rewrite: a pluggable synthesis layer behind the sink nodes (§4.7), adding granular and vocal-tract engines alongside the original additive one, and four more presets - *Wind Noise*, *Azimuth Kinetic*, *Spin Voices (Scale)*, *Vocal Tract* - authored directly as graph patches with no hand-coded precursor to port (§4.5).

5 Evaluation

No performer evaluation has been run yet. This section separates what the current claims actually rest on - implementation-level evidence internal to the system itself - from a protocol for the performer session that would be needed to test the remaining, human-facing parts of those claims. Table 3 summarises this split per claim; the subsections below explain each row.

5.1 Evidence from the Implemented System

Claims (1), (2), and (4) are, at present, supported by implementation-level evidence - the system as built and run by the author during development - rather than by an independent observer. The graph evaluator provisionally adds and removes edges, recomputes a topological order, and drives synthesis every audio block without recompilation or an audio dropout, exercised every time a preset is loaded or a wire is changed. Table 2's per-preset node counts are read directly off the constructed graphs, so claim (2)'s "eleven mappings, one catalog" result is a property of the code, not a reported impression - though the catalog grew alongside those eleven mappings, so its sufficiency for a mapping designed independently of the project remains open (§4.5). Claim (4)'s mount-independence follows from the two-stage calibration derivation holding for any physically valid sensor placement, a property we can state mathematically but have not yet stress-tested by remounting the enclosure at several positions. Claim (3) is the weakest of the four: the facing heuristic runs correctly in every development session, but "correctly" here means

Table 3: Claims, current evidence, and open questions.

Claim	Current evidence	What is still untested
(1) live graph, no recompile	DAG evaluation/rewiring is implemented and exercised continuously during development (§4.5)	whether a performer other than the author can rewire it unaided
(2) 11 mappings as presets	Table 2: all eleven hand-built mappings rebuilt as presets from the same catalog	whether a mapping designed independently of the catalog fits without a new primitive
(3) magnetometer free facing	derivation is implemented and used throughout development sessions	drift has not been measured against a ground-truth heading; bounded to short sessions by assumption, not measurement
(4) mount-agnostic calibration	the correction transform is mount-independent by construction (§6.1)	not yet tested across multiple physical mount positions on hardware

matching the author's own sense of where the staff was pointing, not a logged, ground-truth comparison - drift is bounded to short sessions by assumption, not measurement. No human-subject data has been collected for this instrument, and none of the four claims should be read as validated by user testing until the protocol below, or something like it, has actually been run.

5.2 Planned Performer Evaluation

To test the parts of these claims that are inherently about performer experience rather than code behaviour - whether the graph is rewirable by someone other than its author, whether a Laban-driven preset reads as more physically legible than a simple one, whether the node picker scales to larger patches - we plan a semi-structured session with performers who have some staff, baton, or movement-flow background but no prior exposure to REMORA. The intended structure: a short calibration walkthrough; free play across a small span of existing presets chosen to range from a minimal mapping to a fully Laban-driven one; a guided patching task (retuning an exposed parameter, then rewiring which motion feature drives a sink); and a closing semi-structured interview. Candidate measures include calibration time, time to complete the guided rewire unassisted, think-aloud commentary during free play, a preference judgement between at least two presets with a reason, and any node-picker search difficulties observed or reported. Each measure is chosen to speak to a specific claim in Table 3 rather than to general usability. This protocol - not results - is what we currently have; running it, with informed consent handled as described in the Ethical Statement, is the immediate next step for this work.

6 Discussion

6.1 Advantages and Limitations

The generalisation into a graph buys expressiveness and shareability - eleven previously hand-coded mappings collapse into presets of one small dataflow language - at the cost of a patching step that performers used to fixed instruments do not otherwise have to take. The limitations below are ones the current build genuinely carries, not future work described in present tense.

Network deployment is manual. REMORA joins the host device's own Wi-Fi hotspot rather than a dedicated access point, trading range and interference robustness for a one-step pairing setup. That convenience hides a real cost: the hotspot's Service Set Identifier (SSID)/password and the OSC destination IP address are constants compiled into the firmware. A phone hotspot does not guarantee the same IP across sessions, and pairing with a different host network means a different SSID and password entirely - either case currently requires editing the firmware source and reflashing the microcontroller over the USB before the staff will talk to the plugin again. There is no runtime Wi-Fi configuration, no network discovery of the receiving host, and no feedback if the join fails silently.

Single IMU. All orientation and motion-quality features come from one IMU with no magnetometer fusion; heading is deliberately kept relative to the calibration pose rather than magnetic north, since the magnetometer is unreliable this close to the staff's own electronics and typical stage rigs. That keeps facing free of magnetic interference, but it also means REMORA has no absolute reference at all - any yaw drift accumulated since calibration is invisible and uncorrected, and the facing feature is only ever as good as that session's calibration (§5 bounds claim 3 to session lengths of tens of minutes on this basis, not a measured guarantee). Nothing in the current implementation prevents drift from growing unboundedly over a longer, uninterrupted rehearsal or set, and no re-calibration-on-demand gesture currently exists to correct it mid-performance; ground-truth heading logging would need to be added to the planned performer evaluation before that bound could be tightened.

Graph interface complexity. The node-graph editor is expressive for whoever is building or modifying a mapping, but it exposes that same complexity to a performer who only wants to select and play an existing preset - there is no simplified performance-only view that hides the graph until asked for. The interface currently serves instrument-builders and patch-authors more directly than first-time performers, and whether that cost is acceptable is a question the still-missing user study would need to answer.

Remaining software robustness gaps. A few implementation-level issues would need attention before REMORA is exposed to inputs beyond the author's own testing: incoming OSC messages are read without confirming their argument types match what the receiver expects, dropped or reordered packets are not detected as such, and the handoff of a sensor snapshot from the network thread to the audio thread retries in a tight spin loop rather than degrading gracefully under contention. A related lock has since been addressed rather than left open: graph evaluation used to share a blocking lock with the UI thread editing the live patch, so a rewire mid-performance could stall the audio thread;

NodeGraph::evaluate now takes a ScopedTryLock and simply reuses the previous block's mapping output on contention instead of blocking - the same graceful-degradation pattern the sensor-handoff spin loop above still lacks.

Hardware. The enclosure carries a 2000 mAh LiPo battery, sized to comfortably outlast a single rehearsal or set at the firmware's steady 100 Hz sampling rate, but runtime under REMORA's actual firmware has not been measured: no discharge test has been logged to date. That measurement - ideally several runs to a defined cutoff voltage, reported with variance - is needed before any runtime claim can be made or the battery sized with confidence for a full day of back-to-back performances.

6.2 Future Improvements

Wireless transport. The hotspot-join scheme in §6.1 could be replaced with ESP-NOW: the staff-mounted ESP32 would send its sensor packets directly, peer-to-peer, to a second ESP32 tethered over USB to the host computer, which forwards them to the plugin over serial or a loopback OSC bridge. That removes the SSID/password/IP baked into firmware entirely - ESP-NOW pairs by MAC address rather than joining a network - and sidesteps the "new hotspot means reflash" problem, at the cost of adding a second microcontroller to the setup.

Other extensions. Gesture-triggered mode switching, a merged rest/motion mapping, a two-IMU dual-tip configuration, further Laban Effort mapping targets, machine learning based discrete gesture recognition, and eventually exposing the node graph itself as a shareable/community patch format.

7 Conclusions

REMORA is a wireless, IMU-driven *bô* staff instrument, but its contribution is architectural rather than being one more fixed gesture-to-sound mapping: the same small dataflow language - source, math, and sink node primitives, evaluated as a DAG once per audio block and rewireable without recompiling - reproduces all eleven mapping strategies developed over the course of the project as ordinary, live-editable presets (Table 2), from a minimal tilt-to-pitch patch to the fully Laban-driven *Martial* and *Speed Gate* mappings, grounded in an explicit, code-traceable account of calibration, orientation tracking, and movement-quality extraction (§4.5). The same architecture has since kept absorbing growth without another rewrite: a pluggable synthesis layer added granular and vocal-tract engines alongside the original additive one (§4.7), and four more presets were authored directly as graph patches rather than ported from hand-written code, taking the catalog to fifteen presets in current use.

What remains uncertain is exactly what §5 makes explicit: whether these claims hold for a performer other than the author, and whether the primitive catalog and facing estimate fare under conditions implementation-level evidence alone cannot settle. The most important next step for this work is therefore not another architectural extension but running the performer evaluation protocol outlined in §5.

Acknowledgments

This work was carried out during the first author’s internship at CREATE, Aalborg University, Denmark.

8 Supplementary Material

- **Video demonstration:** the staff, the calibration procedure, and several presets played live - <https://youtu.be/PLACEHOLDER-REMORA-DEMO>
- **Source and design files:** firmware, plugin source, OpenSCAD enclosure files, and all eleven presets described in §4.5 - <https://github.com/TimotheeDvt/REMORA>

9 Ethical Statement

Accessibility. REMORA assumes a range of mobility and two-handed grip strength consistent with swinging a bâ staff; it is not currently adapted for performers who cannot support or swing a 1–1.5 kg full-length staff, and no alternative mounting or reduced-effort variant has been explored - a real limitation of the instrument’s physical form, not merely of the present build. The skill barrier is not uniform across the preset catalog, though: *Simple* asks only for basic tilt and roll, while the Laban-driven presets (*Martial*, *Azimet*) reward accumulated staff-handling skill. Because every preset is authored in the same shared patching language (§4.5), a facilitator can also build a reduced-effort mapping for someone without a staff background - a partial mitigation on the skill axis, not a solution to the physical one above.

Environmental impact. The staff-mounted hardware - a 3D-printed/OpenSCAD-parametrised enclosure, a rechargeable LiPo battery, an ESP32-S2 microcontroller, and an MPU-9250 IMU - is held together with zip ties rather than adhesive (§6.1, The Build), so the enclosure and electronics can be removed, reused, or recycled without damaging the staff. No lifecycle or embodied-carbon analysis of the components has been performed.

Socio-economic fairness. REMORA is built entirely on free/libre and open-source tools - JUCE, PlatformIO, and OpenSCAD - with an inexpensive, widely available microcontroller and IMU, keeping the reproduction cost and toolchain barrier low relative to proprietary DMI environments; the source and design files referenced in Supplementary Material are what make that reproduction actually possible.

Human participants. No human-participant data has been collected to date; all reported observations are the first author’s own, and the performer evaluation outlined in §5 has not yet been conducted. When it is run, participants will give informed consent, be referred to only by anonymised identifiers, and be free to withdraw at any point.

Use of Artificial Intelligence. We disclose that a locally-run large language model (Mistral 7B) was used to help draft and edit portions of this manuscript’s prose under the first author’s direction and review. This disclosure applies only to the writing of this paper: REMORA’s firmware, plugin, and mapping code were designed and written entirely by the first author without AI code generation.

References

- [1] Andrew Belt and VCV. 2017. VCV Rack: Open-Source Virtual Modular Synthesizer. <https://vcvrack.com>. Software, first released September 2017.
- [2] Peter Bennett, Nicholas Ward, Sile O’Modhrain, and Pedro Rebelo. 2007. DAMPER: A Platform for Effortful Interface Development. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. New York, USA. https://www.nime.org/proceedings/2007/nime2007_273.pdf
- [3] Antonio Camurri, Shuji Hashimoto, Matteo Ricchetti, Andrea Ricci, Kenji Suzuki, Riccardo Trocca, and Gualtiero Volpe. 2000. EyesWeb: Toward Gesture and Affect Recognition in Interactive Dance and Music Systems. *Computer Music Journal* 24, 1 (2000), 57–69. doi:10.1162/014892600559182
- [4] Rebecca Fiebrink and Perry R. Cook. 2010. The Wekinator: A System for Real-time, Interactive Machine Learning in Music. In *Late-Breaking Demo, International Society for Music Information Retrieval Conference (ISMIR)*. Utrecht, Netherlands. <https://ismir2010.ismir.net/proceedings/late-breaking-demo-13.pdf>
- [5] Andy Hunt and Marcelo M. Wanderley. 2002. Mapping Performer Parameters to Synthesis Engines. *Organised Sound* 7, 2 (2002), 97–108. doi:10.1017/S1355771802002030
- [6] Joseph Malloch and Marcelo M. Wanderley. 2007. The T-Stick: From Musical Interface to Musical Instrument. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. New York, USA, 66–69. doi:10.1145/1279740.1279751
- [7] Teresa Marrin and Joseph Paradiso. 1997. The Digital Baton: A Versatile Performance Instrument. In *Proceedings of the International Computer Music Conference (ICMC)*. Thessaloniki, Greece. <https://quod.lib.umich.edu/i/icmc/bbp2372.1997.083>
- [8] Teresa Marrin and Rosalind W. Picard. 1998. The “Conductor’s Jacket”: A Device for Recording Expressive Musical Gestures. In *Proceedings of the International Computer Music Conference (ICMC)*. Ann Arbor, MI, USA. <https://www.media.mit.edu/publications/the-conductors-jacket-a-device-for-recording-expressive-musical-gestures-2/>
- [9] Sangbong Nam. 2013. Musical Poi (mPoi). In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. Daejeon, Korea, 148–151. https://www.nime.org/proceedings/2013/nime2013_254.pdf
- [10] Miller Puckette. 1988. The Patcher. In *Proceedings of the International Computer Music Conference (ICMC)*. San Francisco, CA, USA, 420–429.
- [11] Miller Puckette. 1996. Pure Data: Another Integrated Computer Music Environment. In *Proceedings of the Second Intercollege Computer Music Concerts*. Tachikawa, Japan, 37–41.
- [12] Miller Puckette. 2002. Max at Seventeen. *Computer Music Journal* 26, 4 (2002), 31–43. doi:10.1162/014892602320991356
- [13] Jan C. Schacher. 2010. Motion to Gesture to Sound: Mapping for Interactive Dance. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. Sydney, Australia, 250–254. https://www.nime.org/proceedings/2010/nime2010_250.pdf
- [14] Nicholas Ward, Kris Penfield, Sile O’Modhrain, and R. Benjamin Knapp. 2008. A Study of Two Thereminists: Towards Movement Informed Instrument Design. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. Genova, Italy. https://www.nime.org/proceedings/2008/nime2008_117.pdf